

Relatório do Merge Sort

Vitor Kawã Pedrosa da Costa
Universidade do Vale do Itajaí (Univali) – Campus KobraSol

16 de Junho de 2026

Resumo

Este relatório descreve o funcionamento mecânico e a análise de complexidade computacional (notação Big- $\mathcal{O}$) do algoritmo de ordenação *Merge Sort*. O método foi testado na prática por meio de simulações com dados em diferentes estados de organização. As conclusões teóricas são fundamentadas na literatura clássica sobre análise e projeto de algoritmos.

1 Introdução e Funcionamento

O *Merge Sort* foi criado por John von Neumann em 1945 (conforme documentado por Knuth, 1998). É um algoritmo de ordenação que utiliza a técnica de *divisão e conquista*. Em vez de tentar ordenar o vetor inteiro de uma vez, ele quebra o problema em pedaços menores e mais fáceis de resolver.

O algoritmo opera em duas etapas contínuas e recursivas:

1. **Divisão:** O vetor original é cortado ao meio sucessivas vezes, até que sobrem apenas vetores de um único elemento. Logicamente, um vetor que possui apenas um elemento já está ordenado.
2. **Intercalação (*Merge*):** O algoritmo começa a juntar (intercalar) esses pequenos pedaços de volta. Ele compara os elementos dos dois subvetores e os copia na ordem correta para um vetor maior, até reconstruir o tamanho original.

A eficiência deste algoritmo existe porque juntar duas listas que já estão ordenadas é uma operação rápida, que exige apenas uma única passagem pelos dados (custo linear $\mathcal{O}(n)$).

2 Análise de Complexidade (Notação Big- $\mathcal{O}$)

Ao contrário de algoritmos como o *Quicksort*, que podem ficar muito lentos se derem azar na escolha do pivô, o *Merge Sort* tem um comportamento totalmente previsível. A sua divisão simétrica garante a relação matemática:

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n) \quad (1)$$

Aplicando o Teorema Mestre da computação, chegamos aos seguintes limites:

- **Melhor, Médio e Pior Caso de Tempo ($\Theta(n \log n)$):** O tempo de execução será da ordem de $n \log n$ independentemente de como os dados originais estavam misturados. Mesmo que o vetor já esteja ordenado ou que esteja totalmente de trás para frente, o *Merge Sort* fará a mesma quantidade de cortes recursivos e intercalações.
- **Complexidade Espacial (Memória) ($\mathcal{O}(n)$):** Este é o principal defeito do algoritmo. Ele não consegue fazer a ordenação dentro do próprio vetor original (*in-place*). Para que a etapa de intercalação funcione, ele obrigatoriamente precisa criar vetores auxiliares na memória para copiar os dados. O espaço extra necessário é sempre igual ao tamanho dos dados de entrada.

3 Testes e Validação Prática

Para comprovar a teoria, a implementação do *Merge Sort* em C++ foi testada medindo o número de comparações e o número de cópias/movimentações feitas na memória.

1. **Dados já ordenados (Crescente):** O algoritmo demonstrou que não é capaz de perceber que o trabalho já está pronto. Ele realizou as quebras e copiou todos os elementos para os vetores auxiliares da mesma forma.
2. **Dados invertidos (Decrescente):** O desempenho se manteve estável. Enquanto algoritmos mais simples sofrem explosões de tempo $\mathcal{O}(n^2)$ neste cenário, o *Merge Sort* cravou a eficiência imposta por sua regra matemática, executando o mesmo volume de processamento $\mathcal{O}(n \log n)$.
3. **Dados aleatórios:** Validou a estabilidade geral da técnica de *divisão e conquista* perante o uso em condições normais do mundo real.

4 Conclusão

O algoritmo *Merge Sort* continua sendo essencial na ciência da computação devido à sua garantia inquebrável de ser rápido ($\mathcal{O}(n \log n)$) e por ser **estável** (quando há elementos com valores repetidos, ele não altera a ordem original entre eles).

Por outro lado, em sistemas modernos ou dispositivos limitados onde a memória RAM é o maior problema, o *Merge Sort* perde vantagem competitiva. A necessidade de alocar novos vetores para o *merge* (custo $\mathcal{O}(n)$) causa atrasos de leitura no processador (conhecidos como *cache misses*), tornando algoritmos como o *Heap Sort* (que operam *in-place* sem usar espaço extra) melhores escolhas nesse contexto específico.

Referências

- [1] Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley Professional.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.